SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: DSA0109

COURSE NAME: OBJECT ORIENTED PROGRAMMING WITH C++

Q1 – Call by Value, Call by Reference and Call by Pointer

1. Explanation

C++ supports three common methods of passing arguments to functions.

- **Call by Value:** A copy of the original variable is passed to the function. Changes made inside the function do not affect the original variable.
- **Call by Reference:** The function receives a reference to the original variable. Changes made inside the function directly affect the original variable.
- **Call by Pointer:** The address of the original variable is passed to the function. The dereference operator (*) is used to modify its value.

2. Three Implementations

```cpp
#include <iostream>
using namespace std;

// Call by Value
void concessionByValue(double fee, int percent)
{
    fee = fee - (fee * percent / 100);
    cout << "Inside value function: " << fee << endl;
}

// Call by Reference
void concessionByReference(double& fee, int percent)
{
    fee = fee - (fee * percent / 100);
    cout << "Inside reference function: " << fee << endl;
}

// Call by Pointer
void concessionByPointer(double* fee, int percent)
{
    *fee = *fee - (*fee * percent / 100);
    cout << "Inside pointer function: " << *fee << endl;
}
```

```
int main()
{
    double fee1 = 45000;
    double fee2 = 45000;
    double fee3 = 45000;

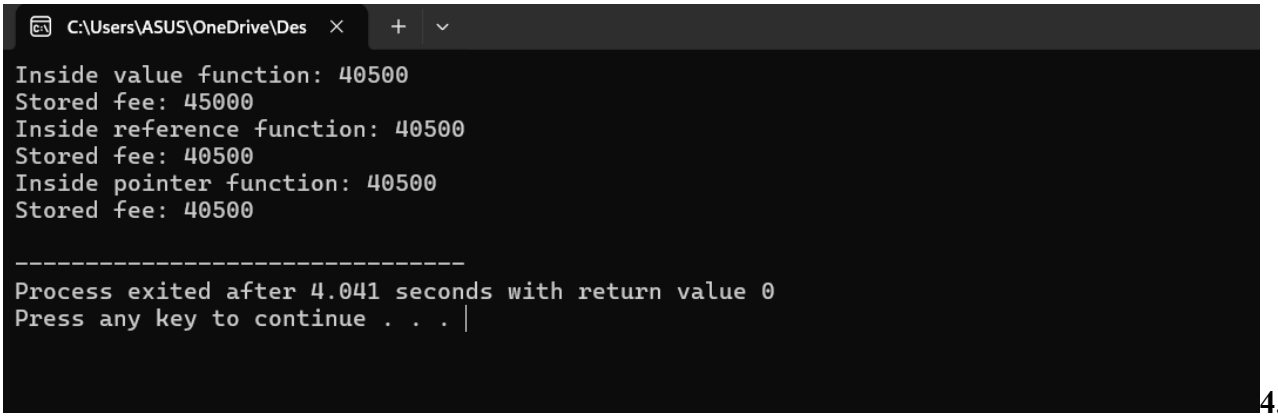
    concessionByValue(fee1, 10);
    cout << "Stored fee: " << fee1 << endl;

    concessionByReference(fee2, 10);
    cout << "Stored fee: " << fee2 << endl;

    concessionByPointer(&fee3, 10);
    cout << "Stored fee: " << fee3 << endl;

    return 0;
}
```

3. Output



4.

Explanation of Parameter Storage

Method	Where parameter is stored	Original fee changes?
Call by Value	Separate local copy	No
Call by Reference	Reference to original variable	Yes
Call by Pointer	Pointer containing original address	Yes

A const reference should be used when a function needs to access a variable without modifying it. It avoids unnecessary copying, especially for large objects.

5. Design Decision

When several concessions apply to one student, passing the fee by reference is suitable because every concession directly updates the stored fee.

For example, a student may receive a merit concession followed by a sports concession. Both operations can update the same fee variable.

Returning the updated fee is another valid approach, especially when the function should calculate a result without directly modifying its input.

6. Predict the Output

Given program:

```
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q)
{
    return p > q ? p : q;
}

void show(int x, int y = 5, int z = 10)
{
    std::cout << x + y + z << ' ';
}

int main()
{
    int a = 1, b = 2, c = 3;

    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;

    std::cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);

    std::cout << '\n';
}
```

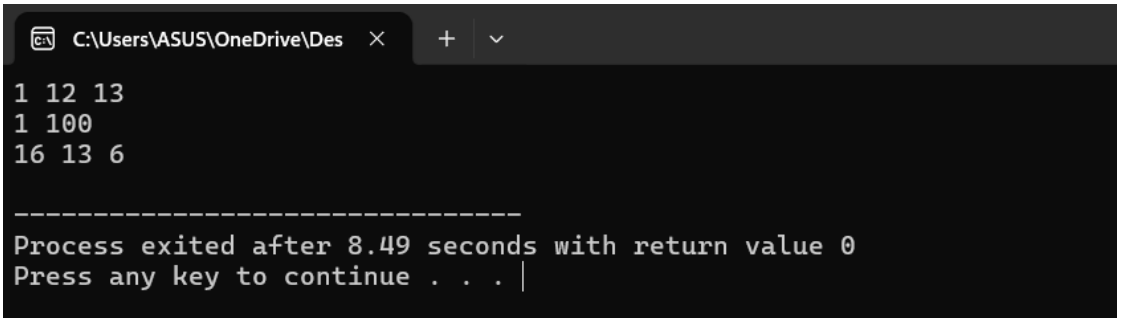
Predicted output:

1 12 13

100 100

16 13 6

Actual Output



Explanation

1. a is passed by value. Its original value remains 1.
2. b is passed by reference. Its value changes from 2 to 12.
3. c is passed by pointer. Its value changes from 3 to 13.
4. bigger(a,c) returns a reference to the larger variable, which is c. Therefore, c becomes 100.
5. show(1) uses default arguments 5 and 10. Result = 16.
6. show(1,2) uses the default value 10 for z. Result = 13.
7. show(1,2,3) uses all three supplied arguments. Result = 6.

7. Complete the Code – TODO 1

```cpp
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
    // TODO 1
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

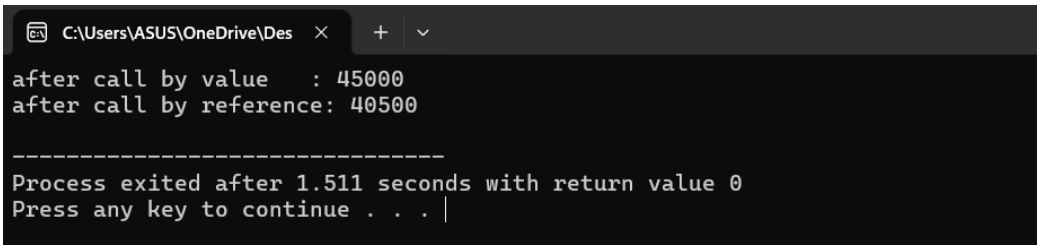
    cout << "after call by value   : "
         << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: "
         << hostelFee << '\n';

    return 0;
}
```

Output:



Why Does Call by Value Leave the Fee at 45000?

The function receives a separate copy of hostelFee. The concession is applied only to that copy. Therefore, the original variable remains unchanged.

In call by reference, the function directly modifies the original variable, so the updated fee becomes 40500.

8. Alternative Concept – Returning the New Fee

If the function returns the updated fee instead of modifying the original variable, it can be written as:

```cpp
long long applyConcession(long long fee, int percent)
{
    return fee - (fee * percent / 100);
}
```

Calling statement:

```
hostelFee = applyConcession(hostelFee, 10);
```

The returned value must be assigned to hostelFee to store the updated fee.

9. Complexity Analysis

Parameter method	Time complexity	Space copied per call
Call by Value	O(1)	Size of the passed object
Call by Reference	O(1)	No object copy
Call by Const Reference	O(1)	No object copy

For a long long, the copied space is constant, O(1).

For a large Student object, call by value copies the entire object, whereas reference and const reference avoid copying it.

Conclusion: Call by value is suitable when the original data must remain unchanged. Call by reference and pointers are suitable when the function must modify the original data.

Q2 – Return by Reference and Its Dangers

1. Explanation

Returning by reference allows a function to return an alias to an existing variable instead of returning a separate copy. It enables the caller to directly modify the original variable.

2. C++ Program

```cpp
#include <iostream>
using namespace std;

// Returns reference to the highest mark
int& highestMark(int marks[], int size)
{
    int maxIndex = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[maxIndex])
        {
            maxIndex = i;
        }
    }
    return marks[maxIndex];
}
// Dangerous function
int& dangerousFunction()
{
    int x = 50;
    return x;
}
int main()
{
    int marks[5] = {75, 89, 92, 84, 78};

    cout << "Highest mark: "
         << highestMark(marks, 5) << endl;
    highestMark(marks, 5) += 2;
    cout << "After moderation: "
         << highestMark(marks, 5) << endl;
    return 0;
}
```

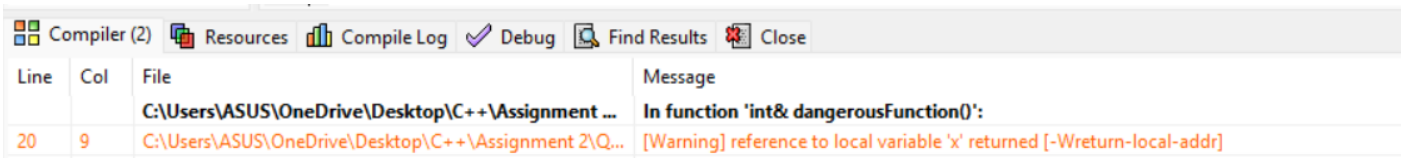
3. Output

```
C:\Users\ASUS\OneDrive\Des
Highest mark: 92
After moderation: 94

--------------------------------
Process exited after 1.546 seconds with return value 0
Press any key to continue . . .
```

The compiler may issue a warning for dangerousFunction() because it returns a reference to a local variable.

The function is deliberately not called.



4. Returning a Reference to a Local Variable

Consider:

```
int& dangerousFunction()
{
    int x = 50;
    return x;
}
```

Here, x is a local variable. Its lifetime ends when the function finishes execution.

Therefore, the returned reference becomes a dangling reference.

Using that reference results in undefined behaviour.

A compiler may produce a warning similar to:

warning: reference to local variable returned

5. Difference Between Reference, Const Reference and Copy

Return type	Description
int&	Returns a modifiable reference
const int&	Returns a reference that cannot be modified through it
int	Returns a separate copy

6. Changed Condition – Using std::vector

When the array is replaced by a std::vector, adding elements may cause reallocation.

During reallocation, the vector may allocate a new memory block and move its existing elements.

As a result, previously obtained references and pointers to vector elements may become invalid.

Therefore, references obtained before vector growth should not be used after a reallocation.

7. Alternative Concept – Returning an Index

```
int highestIndex(int marks[], int size)
{
    int maxIndex = 0;

    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[maxIndex])
        {
            maxIndex = i;
        }
    }

    return maxIndex;
}
```

Calling code:

```
int index = highestIndex(marks, 5);
marks[index] += 2;
```

8. Alternative Concept – Returning a Pointer

```
int* highestPointer(int marks[], int size)
{
    int* max = &marks[0];

    for (int i = 1; i < size; i++)
    {
        if (marks[i] > *max)
        {
            max = &marks[i];
        }
    }

    return max;
}
```

Calling code:

```
int* result = highestPointer(marks, 5);
*result += 2;
```

9. Complexity Analysis

Method	Time complexity	Space complexity
Return by Reference	O(n)	O(1)
Return by Index	O(n)	O(1)
Return by Pointer	O(n)	O(1)

All three methods require O(n) time to find the highest mark.

Conclusion: Returning a reference is convenient for directly modifying an existing array element. Returning an index is often safer when working with a growing vector because the index can be used to access the element again after reallocation, provided the element's logical position is preserved.

Q3 – Default Arguments and Function Overloading

1. Explanation

- **Default Arguments:** Default arguments provide predefined values to function parameters when the caller does not supply them.
- **Function Overloading:** Function overloading allows multiple functions to have the same name but different parameter types or numbers of parameters.

2. Reason for the Compiler Error

The given functions are:

```
double charge(double weight);

double charge(double weight, bool express);

double charge(double weight, bool express = false,
              bool insured = false);
```

The call:

```
charge(2.0);
```

can match all three functions because the third function provides default values for the omitted arguments. Therefore, the call becomes ambiguous, and the compiler cannot select a unique function.

3. Redesigned C++ Program

The redesigned program uses:

- int for weight in grams.
- double for weight in kilograms.
- A Parcel object for parcel details.
- Default arguments for optional express delivery and insurance.

```cpp
#include <iostream>
using namespace std;

class Parcel
{
public:
    double weight;
    bool express;
    bool insured;
    Parcel(double w, bool e = false, bool i = false)
    {
        weight = w;
        express = e;
        insured = i;
    }
};
// Weight in kilograms
double charge(double weight, bool express = false,
              bool insured = false)
{
    double amount = weight * 50;
    if (express)
        amount += 100;
    if (insured)
        amount += 50;
    return amount;
}
// Weight in grams
double charge(int grams)
{
    return (grams / 1000.0) * 50;
}
// Parcel object
double charge(Parcel p)
{
    return charge(p.weight, p.express, p.insured);
}
int main()
{
    cout << "Charge 1: " << charge(2) << endl;
    cout << "Charge 2: " << charge(2.0f) << endl;
    cout << "Charge 3: " << charge(2.0) << endl;
    cout << "Charge 4: " << charge(2.0, true) << endl;
    cout << "Charge 5: " << charge(2.0, false, true) << endl;
    Parcel p(3.0, true, true);
    cout << "Charge 6: " << charge(p) << endl;
    return 0;
}
```

4. Predicted Output

Charge 1: 0.1

Charge 2: 100

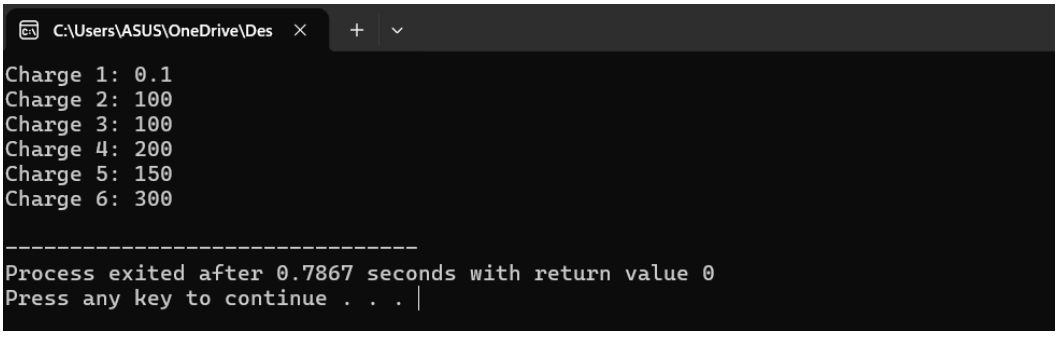
Charge 3: 100

Charge 4: 200

Charge 5: 150

Charge 6: 300

Actual Output



5. Explanation of the Six Function Calls

Function call	Selected function	Reason
charge(2)	charge(int)	Exact integer match
charge(2.0f)	charge(double, bool, bool)	Float promotes to double
charge(2.0)	charge(double, bool, bool)	Exact double match
charge(2.0,true)	Double version	Insurance uses default false
charge(2.0,false,true)	Double version	All arguments supplied
charge(p)	charge(Parcel)	Exact Parcel match

6. How Does the Compiler Select an Overloaded Function?

The compiler considers the available overloaded functions and selects the best match.

The general preference is:

1. Exact match.
2. Standard promotion.
3. Standard conversion.

For example, converting a float to double is a standard floating-point promotion.

7. Why Must Default Arguments Be Given from the Right?

Default arguments must be specified from right to left.

Correct:

```
void display(int a, int b = 10, int c = 20);
```

Incorrect:

```
void display(int a = 10, int b, int c = 20);
```

This rule allows the compiler to identify which arguments have been omitted.

8. Alternative Concept – Using a Parcel Structure

A single Parcel structure can store all the information required for calculating courier charges.

```
struct ParcelInfo
{
    double weight;
    bool express = false;
    bool insured = false;
};
```

The calculation function can accept the structure using a constant reference:

```
double calculateCharge(const ParcelInfo& p)
{
    double amount = p.weight * 50;

    if (p.express)
        amount += 100;

    if (p.insured)
        amount += 50;

    return amount;
}
```

This approach is useful when additional courier options are introduced because new fields can be added to the structure without creating many overloaded functions.

9. Complexity Analysis

Design	Time complexity	Additional space
Function overloading	O(1) calculation	O(1)
Parcel structure	O(1) calculation	O(1) with reference

Function overload resolution happens at compile time and does not introduce runtime selection overhead.

Conclusion: Function overloading handles different input types, while default arguments simplify common calls. A Parcel structure is more convenient when the number of optional courier features increases.

Q4 – Specifying a Class with an Array Inside It

1. Explanation

A class is a user-defined data type that combines data members and member functions.

In this problem, the SavingsAccount class stores the last 10 transactions of a bank account.

The class contains:

Private data members:

- Account number
- Account balance
- Array of 10 transactions
- Transaction count

Public member functions:

- deposit() – Adds money to the account.
- withdraw() – Withdraws money from the account.
- miniStatement() – Displays the last 10 transactions.

2. C++ Program

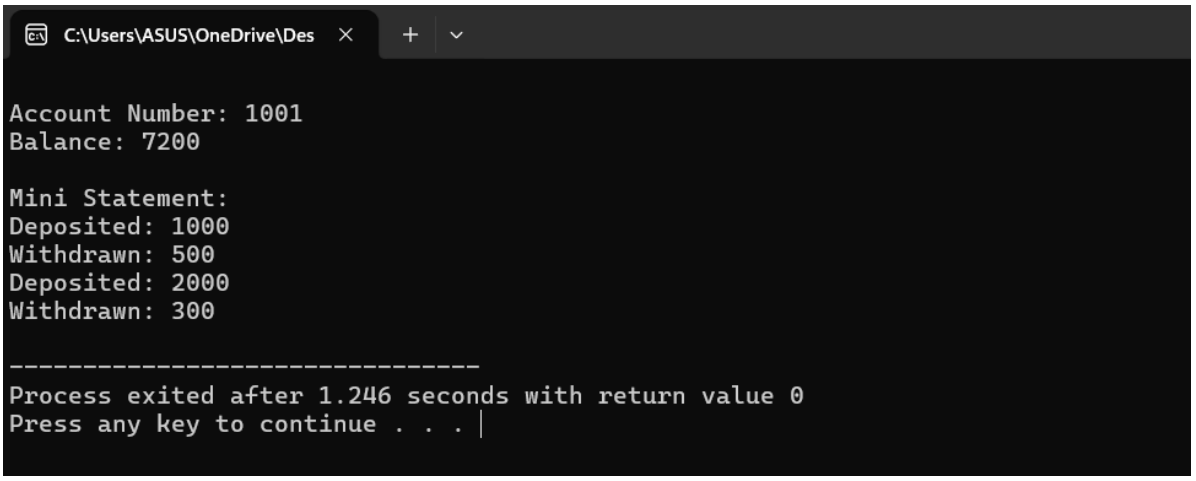
The following program uses a fixed array and defines some member functions inside the class and others outside using the scope resolution operator.

```cpp
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount
{
private:
    int accountNumber;
    double balance;
    string transactions[10];
    int count;
    void addTransaction(string transaction)
    {
        if (count < 10)
        {
            transactions[count] = transaction;
            count++;
        }
        else
        {
            for (int i = 0; i < 9; i++)
            {
                transactions[i] = transactions[i + 1];
            }

            transactions[9] = transaction;
        }
    }
public:
    SavingsAccount(int acc, double amount)
    {
        accountNumber = acc;
        balance = amount;
        count = 0;
    }
    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement()
    {
        cout << "\nAccount Number: " << accountNumber;
        cout << "\nBalance: " << balance << endl;
        cout << "\nMini Statement:\n";
        for (int i = 0; i < count; i++)
        {
            cout << transactions[i] << endl;
        }
    }
};
void SavingsAccount::deposit(double amount)
{
    if (amount > 0)
    {
        balance += amount;
        addTransaction("Deposited: " + to_string(amount));
    }
}
void SavingsAccount::withdraw(double amount)
{
    if (amount > 0 && amount <= balance)
    {
        balance -= amount;
        addTransaction("Withdrawn: " + to_string(amount));
    }
    else
    {
        cout << "Insufficient balance or invalid amount\n";
    }
}
int main()
{
    SavingsAccount account(1001, 5000);
    account.deposit(1000);
    account.withdraw(500);
    account.deposit(2000);
    account.withdraw(300);
    account.miniStatement();
    return 0;
}
```

3. Output



4. Why Must the Array Be Private?

The transaction array should be private because:

1. It prevents direct modification of transaction records.
2. It protects account information.
3. It maintains data security.
4. It allows transactions to be controlled through member functions.

This concept is known as encapsulation.

5. What Happens When the 11th Transaction Arrives?

When the array already contains 10 transactions, a new transaction must be stored by removing the oldest transaction.

Two methods can be used.

- **Method 1 – Shifting the Array**
  - Remove the first transaction.
  - Move every remaining transaction one position to the left.
  - Insert the new transaction at the last position.

The insertion takes O(k) time, where k is the number of stored transactions.

- **Method 2 – Circular Index**

A circular array maintains a starting index that identifies the oldest transaction.

When a new transaction arrives after the array becomes full, the oldest transaction is overwritten and the starting index is updated.

Insertion takes O(1) time.

6. Design Decision – Increasing the Capacity to 50

If the bank wants to store the last 50 transactions instead of 10, the array capacity can be changed from 10 to 50.

However, a circular buffer is preferable because it avoids shifting elements whenever a new transaction arrives.

The circular buffer also requires only a small change in capacity and index management.

7. Alternative Concept – Using std::deque

A deque is a double-ended queue that allows insertion and deletion at both ends.

The following pseudocode demonstrates the required operation:

```
deque<Transaction> transactions;

When a new transaction arrives:
    if size is equal to 10:
        remove the front transaction

    insert the new transaction at the back
```

The operations push_back() and pop_front() allow the program to maintain the latest 10 transactions without manually shifting the elements.

8. Complexity Analysis

Let k represent the maximum number of transactions.

Operation	Fixed Array with Shifting	Circular Buffer	Deque
Insert when full	O(k)	O(1)	O(1) amortized
Display transactions	O(k)	O(k)	O(k)
Space complexity	O(k)	O(k)	O(k)

Conclusion: A fixed array is suitable for a small transaction history. A circular buffer or deque is more efficient when transactions are added frequently because it avoids repeated shifting.

Q5 – Static Members and Arrays of Objects

1. Explanation

A static data member belongs to the entire class rather than to individual objects.

Only one copy of a static data member exists, and all objects of the class share it.

In the bus ticket booking system, the number of tickets issued must be shared by all ticket objects.

The following members are required:

Member	Purpose
issued	Stores the number of seats issued
CAPACITY	Stores the maximum number of seats

count()	Returns the number of tickets issued
seat	Stores an individual ticket's seat number
name	Stores the passenger's name

2. Static Data Member Declaration and Definition

Inside the class:

```
static int issued;
```

Outside the class:

```
int BusTicket::issued = 0;
```

A static data member must be defined outside the class in this program so that storage is provided for it.

3. Why Cannot a Static Member Function Use this?

A static member function belongs to the class rather than to an individual object.

Therefore:

- It does not have a this pointer.
- It cannot directly access non-static data members.
- It can access static data members.
- It can be called using the class name.

Example:

```
BusTicket::count();
```

4. Complete the Code – TODO 1 and TODO 2

Completed program from the PDF:

```
#include <iostream>
#include <string>
using namespace std;
class BusTicket
{
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    string name;
public:
    explicit BusTicket(const string& n) : seat(0), name(n)
    {
        // TODO 1
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
```

```
        // TODO 2
        return issued;
    }
    void print() const
    {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat)
                      : string("Waiting"))
             << '\n';
    }
};
int BusTicket::issued = 0;
int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };
    for (const BusTicket& t : trip)
    {
        t.print();
    }
    cout << "Issued: " << BusTicket::count() << '\n';
    return 0;
}
```

5. Expected Output

Asha: Seat 1

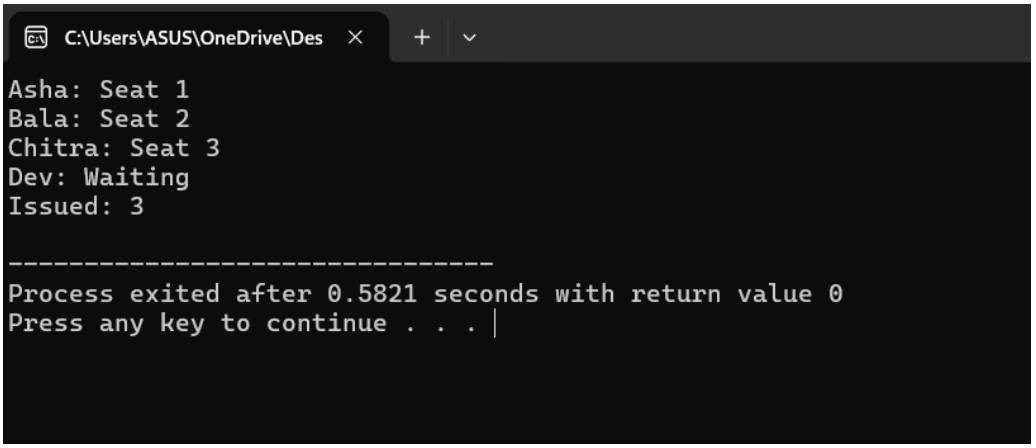
Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Actual Output



Explanation

The bus has a capacity of three seats.

1. Asha receives seat number 1.
2. Bala receives seat number 2.

3. Chitra receives seat number 3.
4. Dev cannot receive a seat because the bus is full. Therefore, Dev is placed on the waiting list.

The static variable issued maintains the total number of seats issued.

6. Creating an Array of Objects

An array of objects contains multiple objects belonging to the same class.

Example:

```
BusTicket trip[4] = {
    BusTicket("Asha"),
    BusTicket("Bala"),
    BusTicket("Chitra"),
    BusTicket("Dev")
};
```

Each object is initialized through the constructor.

When the objects are created inside a loop or function, the constructor executes for every newly created object.

The static counter continues to maintain its value even after local objects are destroyed.

7. Predict the Output

Given program from the PDF:

```
#include <iostream>

class Token
{
    static int next;
    int id;
public:
    Token()
    {
        id = ++next;
    }
    int get() const
    {
        return id;
    }
    static int issued()
    {
        return next - 100;
    }
};

int Token::next = 100;
int main()
{
    Token counter[3];

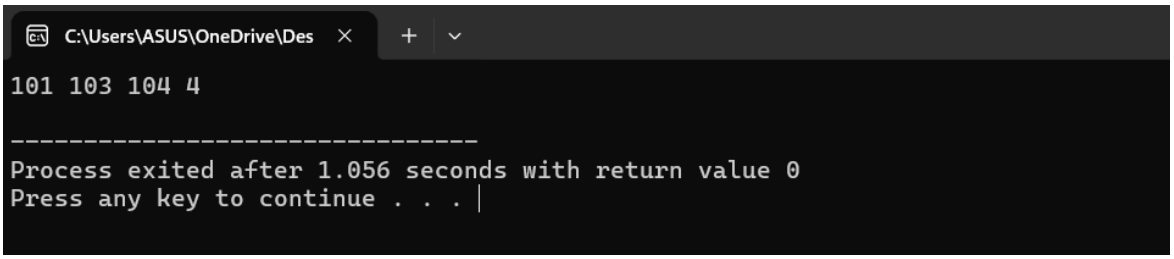
    Token extra;

    std::cout << counter[0].get() << ' '
              << counter[2].get() << ' '
              << extra.get() << ' '
              << Token::issued() << '\n';
}
```

Predicted output:

101 103 104 4

Actual Output



Explanation

The static variable next is initialized to 100.

When the three objects in the array are created, their IDs become:

- First object: 101
- Second object: 102
- Third object: 103

The extra object receives ID 104.

The issued() function returns:

$$104 - 100 = 4$$

Therefore, the constructor executes four times.

8. Creating Tickets Inside a Function Called Twice

Consider the following code:

```
void createTickets()
{
    BusTicket a("Student1");
    BusTicket b("Student2");
}
```

When this function is called twice, four ticket objects are created.

The static counter continues from its previous value because it belongs to the class.

Destroying the local objects does not reset the static counter or automatically release their issued seat numbers.

If the bus capacity has already been reached, subsequent tickets will be placed on the waiting list.

9. Alternative Concept – Global Variable and TicketOffice Object

Global variable approach:

A global variable can maintain the number of issued tickets.

```
int issued = 0;
```

Every ticket constructor can update this variable.

However, global variables can be accessed from different parts of the program, reducing encapsulation.

➢ **TicketOffice object approach:**

A separate TicketOffice class can maintain the seat count.

Each BusTicket object can receive a reference to the TicketOffice object.

This approach is useful when two buses operate independently because each TicketOffice object can maintain a separate counter.

10. Complexity Analysis

Design	Time Complexity	Space Complexity
Static counter	O(1)	O(1) shared counter
Global counter	O(1)	O(1) global counter
TicketOffice object	O(1)	O(1) per office

Conclusion: Static members are useful when information must be shared among all objects of a class. They provide better encapsulation than global variables. A separate TicketOffice object is more suitable when multiple buses need independent seat management.

Q6 – Friend Functions and Inline Functions

1. Explanation

A friend function is a non-member function that is allowed to access the private and protected members of a class.

An inline function is a function for which the compiler may replace the function call with the actual function body, reducing function-call overhead.

The compiler decides whether to perform this substitution. The inline keyword does not guarantee it.

2. Complete the Code – TODO 1 and TODO 2

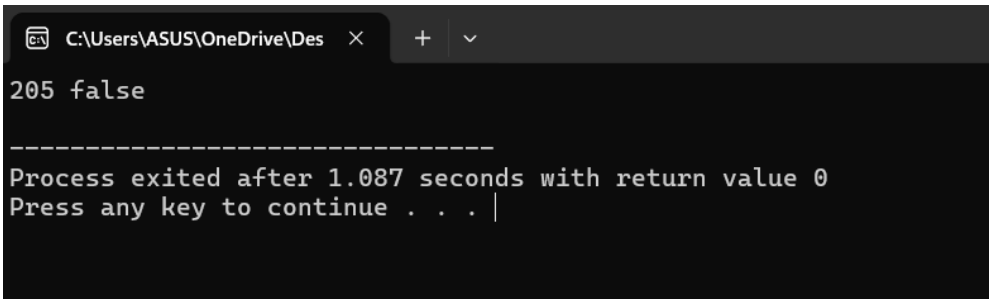
```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time
{
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
// TODO 1
int minutesBetween(const Time& a, const Time& b)
{
    int time1 = a.h * 60 + a.m;
    int time2 = b.h * 60 + b.m;
    return abs(time1 - time2);
}
```

```cpp
// TODO 2
bool earlier(const Time& a, const Time& b)
{
    if (a.h < b.h)
        return true;
    if (a.h == b.h && a.m < b.m)
        return true;
    return false;
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha
         << earlier(arrival, departure) << '\n';
    return 0;
}
```

3. Output



4. Explanation

The Time class contains two private data members:

- h – stores the hour.
- m – stores the minutes.

The friend function minutesBetween() accesses these private members.

The time is converted into minutes using:

$$\text{Total Minutes} = (h \times 60) + m$$

For departure time 9:45:

$$(9 \times 60) + 45 = 585$$

For arrival time 13:10:

$$(13 \times 60) + 10 = 790$$

Difference:

$$790 - 585 = 205$$

Therefore, the difference between the two times is 205 minutes.

The earlier() function checks whether the first time occurs before the second time.

Since 13:10 is later than 9:45, the function returns false.

5. Correctness Tests

The following test cases verify the comparison of times on either side of noon and the same time twice.

First Time	Second Time	Difference	Earlier
9:45	13:10	205	True
13:10	9:45	205	False
12:00	12:00	0	False
11:30	12:30	60	True
12:30	11:30	60	False

These tests confirm that the functions correctly calculate the time difference and compare the two times.

6. Why Is a Friend Function Needed?

A friend function is useful when an operation involves two objects equally.

In this problem, the functions compare two Time objects.

A friend function can directly access the private members of both objects without requiring public getter functions.

7. Difference Between Friend Function and Member Function

Friend Function	Member Function
Not a member of the class	Belongs to the class
Called like a normal function	Called using an object
Can access private members through friendship	Can access private members of its object
Does not have a this pointer	Has a this pointer
Can work with two objects equally	Usually operates on one object

8. Alternative Concept – Using Member Functions

The comparison can also be implemented using member functions.

```
int minutesTo(const Time& b) const;

bool isEarlier(const Time& b) const;
```

Calling code:

```
departure.minutesTo(arrival);

arrival.isEarlier(departure);
```

Another alternative is to provide public getter functions for the private hour and minute values. A normal non-member function can then perform the comparison using those getters.

However, getters expose additional access methods, whereas friend functions provide direct access only to the functions explicitly declared as friends.

9. Inline Performance Investigation

The assignment requires measuring 10,000,000 function calls with and without inline under two optimization levels.

The following benchmark can be used.

```cpp
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

inline int inlineAdd(int a, int b)
{
    return a + b;
}
int normalAdd(int a, int b)
{
    return a + b;
}
int main()
{
    volatile long long result = 0;

    auto start = high_resolution_clock::now();

    for (int i = 0; i < 10000000; i++)
    {
        result = result + inlineAdd(i, 2);
    }
    auto end = high_resolution_clock::now();
    cout << "Inline Time: "
         << duration_cast<microseconds>(
              end - start).count()
         << " microseconds\n";
    start = high_resolution_clock::now();
    for (int i = 0; i < 10000000; i++)
    {
        result = result + normalAdd(i, 2);
    }
    end = high_resolution_clock::now();
    cout << "Normal Time: "
         << duration_cast<microseconds>(
              end - start).count()
         << " microseconds\n";
    return 0;
}
```

Compilation Commands

Without optimization (-O0):

```
g++ -std=c++17 -O0 benchmark.cpp -o benchmark
```

With optimization (-O2):

```
g++ -std=c++17 -O2 benchmark.cpp -o benchmark
```

Run the program for both compilation commands and record the actual execution times.

Observation Table

Optimization	Inline Function	Normal Function
-O0	Record actual time	Record actual time
-O2	Record actual time	Record actual time

At -O0, function-call overhead may be more noticeable. At -O2, the compiler may optimize both functions, making their execution times similar.

The actual results depend on the compiler and computer.

10. Deployment Decision

Friend functions should remain when two objects need equal access to private data.

Member functions are preferable when an operation naturally belongs to one object.

Making private members public is not recommended because it weakens encapsulation and allows unrestricted modification of the data.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Friend functions	O(1)	O(1)
Member functions	O(1)	O(1)
Public getters	O(1)	O(1)
Inline functions	O(1)	O(1)

Conclusion: Friend functions are useful for operations involving two objects equally. Inline functions may reduce function-call overhead for small operations. The final implementation should maintain encapsulation while considering actual performance measurements.

Q7 – Diagnose, Redesign and Validate

Selected program: Q4 – Savings Account Mini-Statement

The assignment asks you to select ONE program from Q1–Q6, identify a limitation, redesign it, implement the revised solution and compare both versions.

A. Diagnose the Limitation

Original Design

The original SavingsAccount program uses a fixed-size array to store the last 10 transactions.

When the 11th transaction arrives, the oldest transaction is removed by shifting the remaining elements one position to the left.

Identified Limitation

The main limitation is the time required to insert a new transaction when the array is full.

For example:

- The oldest transaction must be removed.
- The remaining nine transactions must be shifted.
- The new transaction must be inserted at the last position.

This operation requires multiple assignments for every new transaction.

Evidence

Transaction	Operation
1st to 10th	Direct insertion
11th	Shift 9 elements
12th	Shift 9 elements
13th	Shift 9 elements

Therefore, repeated shifting becomes inefficient when the transaction capacity increases.

B. Define the Design Objective

The objective is to redesign the SavingsAccount program using a circular array.

The redesigned system should:

1. Store the latest 10 transactions.
2. Automatically discard the oldest transaction.
3. Avoid shifting existing transactions.
4. Maintain the correct order when displaying transactions.
5. Reduce insertion time from O(k) to O(1).

C. Design the Revised Solution

Proposed Design – Circular Buffer

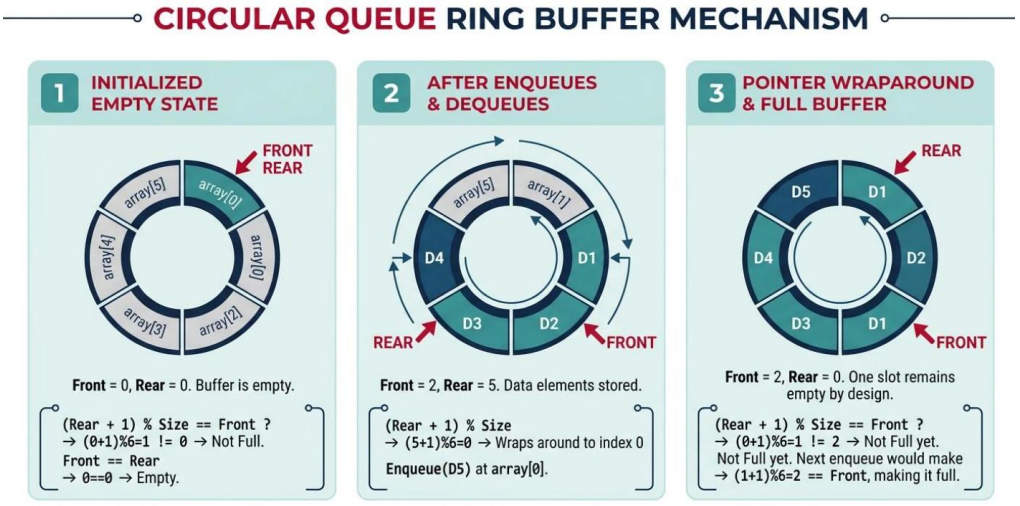
A circular buffer is a fixed-size array in which the last position connects back to the first position.

It uses a starting index to identify the oldest transaction.

When a new transaction arrives:

1. Calculate the insertion position.
2. Insert the transaction.
3. If the array is not full, increase the count.
4. If the array is full, overwrite the oldest transaction and update the starting index.

Circular Buffer Representation



Revised C++ Program

```cpp
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount
{
private:
    int accountNumber;
    double balance;
    string transactions[10];
    int start;
    int count;
public:

    SavingsAccount(int acc, double amount)
    {
        accountNumber = acc;
        balance = amount;
        start = 0;
        count = 0;
    }
    void addTransaction(string transaction)
    {
        int position = (start + count) % 10;
        transactions[position] = transaction;
        if (count < 10)
        {
            count++;
        }
        else
        {
            start = (start + 1) % 10;
        }
    }
    void deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            addTransaction(
                "Deposit: " + to_string(amount)
            );

        }
    }
    void withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            addTransaction(
                "Withdraw: " + to_string(amount)
            );
        }
    }
    void miniStatement()
    {
        cout << "Account Number: "
             << accountNumber << endl;
        cout << "Balance: "
             << balance << endl;
        cout << "\nLast 10 Transactions:\n";
        for (int i = 0; i < count; i++)
        {
            int position = (start + i) % 10;
            cout << transactions[position] << endl;
        }
    }
};
int main()
{
    SavingsAccount account(1001, 5000);
    for (int i = 1; i <= 12; i++)
    {
        account.deposit(100);
    }
    account.miniStatement();
    return 0;
}
```

Output



The program inserts 12 transactions but retains only the latest 10. The first two transactions are automatically discarded.

D. Predict Before Implementing

The redesigned circular buffer is expected to perform better than the original array when inserting transactions after the array becomes full.

Prediction:

- Original design requires shifting nine elements for every new transaction after reaching capacity.
- Redesigned design overwrites the oldest transaction and updates the starting index.
- The redesigned implementation should reduce insertion time from O(k) to O(1).

E. Implement and Validate the Redesign

1. Comparison of Original and Redesigned Programs

Parameter	Original Program	Redesigned Program
Data structure	Fixed array	Circular array
Insertion method	Shifting	Overwriting
Insertion complexity	O(k) when full	O(1)
Display complexity	O(k)	O(k)
Space complexity	O(k)	O(k)
Stores latest 10	Yes	Yes

2. Validation Test Cases

Test case	Number of transactions	Expected result
Test 1	5	All 5 stored

Test 2	10	All 10 stored
Test 3	11	First transaction removed
Test 4	12	First 2 transactions removed
Test 5	15	Latest 10 transactions retained

3. Performance Measurement

Execute both versions with the same input sizes.

Number of transactions	Original array	Circular buffer
1,000	Record time	Record time
10,000	Record time	Record time
100,000	Record time	Record time
1,000,000	Record time	Record time

The execution time should be measured on your computer. Do not fill this table with estimated timings.

F. Defend the Final Design

The circular buffer is a better choice for a transaction system where new transactions are added frequently.

- **Advantages**
  1. Avoids shifting existing elements.
  2. Provides constant-time insertion.
  3. Uses fixed memory.
  4. Automatically replaces the oldest transaction.
  5. Maintains the latest transaction history.
- **Disadvantages**
  1. Requires additional index management.
  2. Displaying transactions requires circular index calculations.
  3. The implementation is slightly more complex than a normal array.

When Is the Original Design Preferable?

The original array with shifting is suitable when the number of transactions is small and insertions are infrequent.

It is easier to implement and understand.

Final Conclusion

The redesigned circular buffer addresses the insertion-time limitation of the original SavingsAccount program. It reduces insertion complexity from O(k) to O(1) while maintaining the same space complexity of O(k). Therefore, the circular buffer is more suitable for applications that frequently update transaction records.